HackDNA – Hack the Cookie

Challenge Overview

This challenge focuses on insecure client-side trust and cookie manipulation. Web applications commonly use cookies to maintain session state, authentication information, and user preferences.

When sensitive authorization logic is stored directly inside client-side cookies without proper integrity protection, attackers can manipulate values to escalate privileges or bypass authentication controls.

Objective

Analyze and manipulate browser cookies to gain elevated privileges or unauthorized access.

Understanding Cookies

Cookies are small pieces of data stored in the browser and sent with HTTP requests.

Common uses include:

- Session management
- Authentication
- User preferences
- Tracking
- State persistence

Example cookie:

```
Cookie: role=user
```

If applications trust cookie values directly, attackers may tamper with them.

Reconnaissance

Open browser developer tools.

```
F12 → Application → Cookies
```

or

```
Storage → Cookies
```

Inspect available cookie values.

Potential indicators:

```
role=user
admin=false
isAdmin=0
access=basic
```

These values may control authorization.

Initial Enumeration

Observe application behavior while logged in.

Questions to investigate:

- Does the cookie contain plaintext values?
- Is the cookie encoded?
- Is there any signature validation?
- Does changing the value alter permissions?

Example vulnerable cookie:

```
user=guest
```

or:

```
role=user
```

Cookie Manipulation

Modify the cookie value manually.

Example:

```
role=admin
```

or:

```
admin=true
```

Refresh the application after editing the cookie.

If the application trusts the modified value without verification, administrative functionality may become accessible.

Encoded Cookies

Applications sometimes Base64-encode cookies.

Example encoded value:

```
eyJyb2xlIjoidXNlciJ9
```

Decode using:

```
echo 'eyJyb2xlIjoidXNlciJ9' | base64 -d
```

Decoded result:

```
{"role":"user"}
```

Modify the value:

```
{"role":"admin"}
```

Re-encode:

```
echo '{"role":"admin"}' | base64
```

Replace the cookie with the new encoded value.

JWT-Based Cookies

Some applications use JSON Web Tokens (JWTs).

Structure:

```
HEADER.PAYLOAD.SIGNATURE
```

Example decoded payload:

```
{
  "user":"guest",
  "role":"user"
}
```

If JWT signature verification is weak or disabled, attackers may modify the payload and forge administrative tokens.

Exploitation

After modifying the cookie:

- Refresh the page
- Access restricted endpoints
- Attempt admin panel access
- Test privileged functionality

Successful exploitation demonstrates insecure trust of client-side authorization data.

Root Cause

The vulnerability occurs because authorization decisions are based on user-controlled data.

Applications should never trust:

- Client-side role values
- Unsigned cookies
- Weakly signed tokens
- Editable authorization parameters

All sensitive authorization logic must be validated server-side.

Security Impact

Cookie tampering vulnerabilities can lead to:

- Privilege escalation
- Authentication bypass
- Administrative access
- Session hijacking
- Sensitive data exposure
- Full account compromise

Detection Techniques

Manual Testing

- Inspect cookies
- Modify values
- Observe authorization changes
- Decode Base64 values
- Analyze JWT payloads

Proxy-Based Testing

Using Burp Suite:

```
Proxy → Intercept → Modify Cookie Header
```

Using browser extensions:

- EditThisCookie
- Cookie Editor

Example Vulnerable Scenario

Server logic:

```
if request.cookies.get("role") == "admin":
    grant_admin_access()
```

An attacker simply changes:

```
role=user
```

to:

```
role=admin
```

and gains unauthorized access.

Secure Design Principles

Server-Side Authorization

Authorization decisions must be enforced on the server.

Use Signed Tokens

Cookies containing sensitive data should be:

- Signed
- Encrypted
- Integrity protected

Validate Sessions Securely

Use server-managed sessions instead of client-controlled roles.

Apply HttpOnly and Secure Flags

Example:

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Strict
```

Implement Least Privilege

Users should only receive the minimum required permissions.

Common Real-World Issues

Developers frequently expose:

- Role identifiers
- User IDs
- Access levels
- Feature flags
- Session metadata

inside editable cookies.

Poor JWT validation is also a recurring issue in modern web applications.

Key Takeaway

Client-side data is fully under attacker control.

Any authorization mechanism relying solely on browser-stored values is fundamentally insecure.

Vulnerability Classification

- CWE-565: Reliance on Cookies without Validation and Integrity Checking
- CWE-602: Client-Side Enforcement of Server-Side Security
- CWE-639: Authorization Bypass Through User-Controlled Key

- OWASP A01:2021 – Broken Access Control

Conclusion

Cookie manipulation attacks demonstrate the dangers of trusting client-side authorization data. Attackers routinely inspect and tamper with cookies during web application assessments.

Proper server-side authorization, signed session management, and secure token validation are essential to prevent privilege escalation vulnerabilities.